

Telecom Usage & Profile Enrichment Analytics – 27 Solutions

Simple PySpark solutions without the F. prefix.

1. define_schema()

```
def define_schema():
    return StructType([
        StructField("event_id", StringType(), True),
        StructField("user_id", IntegerType(), True),
        StructField("plan", StringType(), True),
        StructField("city", StringType(), True),
        StructField("event_date", StringType(), True),
        StructField("data_mb", DoubleType(), True),
        StructField("voice_minutes", DoubleType(), True),
        StructField("sms_count", IntegerType(), True),
        StructField("roaming", StringType(), True),
        StructField("device_type", StringType(), True),
        StructField("dropped_calls", IntegerType(), True),
        StructField("latency_ms", IntegerType(), True)
    ])
```

2. load_data()

```
def load_data(spark, path, schema):
    return spark.read.csv(path, header=True, schema=schema)
```

3. parse_event_date()

```
def parse_event_date(df):
    return df.withColumn("event_date", to_date("event_date"))
```

4. add_total_activity()

```
def add_total_activity(df):
    return df.withColumn("total_activity",
                        col("voice_minutes") + col("sms_count"))
```

5. add_usage_score()

```
def add_usage_score(df):
    return df.withColumn(
        "usage_score",
        col("data_mb") +
        (col("voice_minutes") * 0.5) +
        (col("sms_count") * 0.1)
    )
```

6. filter_roaming_events()

```
def filter_roaming_events(df):
    return df.filter(col("roaming") == "Y")
```

7. top_n_users_by_data()

```
def top_n_users_by_data(df, n):
```

```

return (
    df.groupBy("user_id")
      .agg(sum("data_mb").alias("total_data_mb"))
      .orderBy(col("total_data_mb").desc(),
                col("user_id").asc())
      .limit(n)
)

```

8. avg_latency_by_city()

```

def avg_latency_by_city(df):
    return (
        df.groupBy("city")
          .agg(avg("latency_ms").alias("avg_latency_ms"))
    )

```

9. most_used_plan()

```

def most_used_plan(df):
    result = (
        df.groupBy("plan")
          .agg(count("*").alias("total_count"))
          .orderBy(col("total_count").desc(),
                    col("plan").asc())
          .limit(1)
          .collect()
    )
    return result[0]["plan"]

```

10. dropped_call_rate_by_plan()

```

def dropped_call_rate_by_plan(df):
    return (
        df.groupBy("plan")
          .agg(
              count("*").alias("total_calls"),
              sum("dropped_calls").alias("dropped_total")
          )
          .withColumn(
              "dropped_call_rate",
              col("dropped_total") / col("total_calls")
          )
          .select("plan", "dropped_call_rate")
    )

```

11. high_latency_events()

```

def high_latency_events(df, threshold):
    return df.filter(col("latency_ms") > threshold)

```

12. count_users_per_city()

```

def count_users_per_city(df):
    return (
        df.groupBy("city")
          .agg(countDistinct("user_id").alias("user_count"))
    )

```

13. daily_data_trend()

```

def daily_data_trend(df):
    return (
        df.groupBy("event_date")

```

```

        .agg(sum("data_mb").alias("daily_data_mb"))
        .orderBy(col("event_date").asc())
    )

```

14. top_device_by_usage_score()

```

def top_device_by_usage_score(df):
    df = add_usage_score(df)

    result = (
        df.groupBy("device_type")
            .agg(avg("usage_score").alias("avg_usage_score"))
            .orderBy(col("avg_usage_score").desc(),
                    col("device_type").asc())
            .limit(1)
            .collect()
    )
    return result[0]["device_type"]

```

15. list_cities()

```

def list_cities(df):
    df = df.select("city").distinct().collect()

    cities = []
    for row in df:
        cities.append(row["city"])

    return sorted(cities)

```

16. events_in_date_range()

```

def events_in_date_range(df, start, end):
    return df.filter(
        col("event_date").between(
            to_date(lit(start)),
            to_date(lit(end))
        )
    )

```

17. flag_network_risk()

```

def flag_network_risk(df, latency_threshold, drop_threshold):
    return df.withColumn(
        "is_risky",
        (col("latency_ms") > latency_threshold) |
        (col("dropped_calls") > drop_threshold)
    )

```

18. top_n_risky_users()

```

def top_n_risky_users(df, n):
    if "is_risky" not in df.columns:
        df = flag_network_risk(df, 150, 1)

    return (
        df.filter(col("is_risky") == True)
            .groupBy("user_id")
            .agg(count("*").alias("risky_count"))
            .orderBy(col("risky_count").desc(),
                    col("user_id").asc())
            .limit(n)
    )

```

19. avg_data_per_plan()

```
def avg_data_per_plan(df):
    return (
        df.groupBy("plan")
            .agg(avg("data_mb").alias("avg_data_mb"))
    )
```

20. get_heaviest_user()

```
def get_heaviest_user(df):
    result = (
        df.groupBy("user_id")
            .agg(sum("data_mb").alias("total_data_mb"))
            .orderBy(col("total_data_mb").desc(),
                    col("user_id").asc())
            .limit(1)
            .collect()
    )

    return (
        result[0]["user_id"],
        result[0]["total_data_mb"]
    )
```

21. define_profile_schema()

```
def define_profile_schema():
    return StructType([
        StructField("user_id", IntegerType(), True),
        StructField("first_name", StringType(), True),
        StructField("last_name", StringType(), True),
        StructField("full_name", StringType(), True),
        StructField("plan_id", StringType(), True)
    ])
```

22. define_plan_schema()

```
def define_plan_schema():
    return StructType([
        StructField("plan_id", StringType(), True),
        StructField("plan_name", StringType(), True)
    ])
```

23. load_inline_profile_data()

```
def load_inline_profile_data(spark, profile_schema, plan_schema):
    profiles_rows = [
        (201, "Aarav", "Iyer", None, "P1"),
        (202, "Diya", "Sharma", "Diya Sharma", "P2"),
        (203, "Kabir", "Mehta", None, "P2"),
        (204, "Meera", "Nair", None, "P3"),
        (205, "Rohan", "Singh", "Rohan S.", "P3"),
        (206, "Sara", "Khan", None, "P99")
    ]

    plans_rows = [
        ("P1", "Basic"),
        ("P2", "Plus"),
        ("P3", "Premium")
    ]

    profiles_df = spark.createDataFrame(profiles_rows, schema=profile_schema)
    plans_df = spark.createDataFrame(plans_rows, schema=plan_schema)

    return profiles_df, plans_df
```

24. join_profile_plan()

```
def join_profile_plan(profiles_df, plans_df):  
    return profiles_df.join(plans_df, "plan_id", "left")
```

25. enrich_full_name()

```
def enrich_full_name(profile_joined_df):  
    return profile_joined_df.withColumn(  
        "full_name",  
        when(  
            col("full_name").isNull(),  
            concat_ws(" ", col("first_name"), col("last_name"))  
        ).otherwise(col("full_name"))  
    )
```

26. add_event_epoch_seconds()

```
def add_event_epoch_seconds(df):  
    return df.withColumn(  
        "event_epoch_seconds",  
        unix_timestamp("event_date")  
    )
```

27. add_event_ts_from_epoch()

```
def add_event_ts_from_epoch(epoch_df):  
    return epoch_df.withColumn(  
        "event_ts",  
        from_unixtime(col("event_epoch_seconds")).cast("timestamp")  
    )
```